Лабораторна робота №10

Тема: Ефективне керування унікальними даними за допомогою `Map` та `Set`.

Мета: Навчитися використовувати структури `Map` для зберігання пар «ключ-значення» та `Set` для роботи з унікальними елементами; порівняти їхню продуктивність та функціональність зі звичайними об'єктами та масивами.

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Колекція Set:

Множина унікальних значень Set − це вбудований об'єкт, який дозволяє зберігати унікальні значення будь-якого типу (як примітиви, так і об'єкти).

- *Механізм перевірки унікальності.* Set використовує алгоритм SameValueZero, який схожий на суворе порівняння (= = =), але вважає, що NaN дорівнює NaN.
- *Продуктивність.* Пошук елемента в Set за допомогою методу `.has()` виконується значно швидше O(1), ніж пошук у масиві через `.includes()` O(n), оскільки Set використовує хеш-таблиці.
- *Перетворення.* Найшвидший спосіб очистити масив від дублікатів − це комбінація `Set` та оператора `spread (...)`:

```
JavaScript
const  unique = [...new Set([1, 2, 2, 3])]; // [1, 2, 3]
```

ОПЕРАЦІЇ НАД МНОЖИНАМИ (SET OPERATIONS)

У JavaScript об'єкт `Set` не має вбудованих методів для математичних операцій (таких як об'єднання чи перетин) «з коробки», проте їх легко реалізувати за допомогою методів масивів та конструктора `new Set()`.

1. Об'єднання (Union)

Створення нової множини, що містить усі елементи з обох множин без дублікатів. Об'єднати обидві множини в один масив і передати його в конструктор `Set`.

Приклад:

```
JavaScript
const setA = new Set(['Ford', 'Toyota']);
const setB = new Set(['Tesla', 'Ford']);
const union = new Set([...setA, ...setB]);
// Результат: Set {'Ford', 'Toyota', 'Tesla'}
```

2. Перетин (Intersection)

Створення множини, що містить лише ті елементи, які є одночасно і в першій, і в другій множині. Відфільтрувати першу множину, залишивши лише ті елементи, які присутні в другій (`has`).

Приклад:

```
JavaScript
```

```
const intersection = new Set([...setA].filter(x => setB.has(x)));
// Результат: Set {'Ford'}
```

3. Різниця (Difference)

Створення множини, що містить елементи, які є в першій множині, але відсутні в другій. Відфільтрувати першу множину, видаливши ті елементи, що є в другій.

Приклад:

```
JavaScript
const difference = new Set([...setA].filter(x => !setB.has(x)));
// Результат: Set {'Toyota'}
```

4. Симетрична різниця (Symmetric Difference)

Створення множини, що містить елементи, які є в одній із множин, але не в обох одночасно (унікальні для кожної сторони).

Приклад:

```
JavaScript
const symDiff = new Set(
[...setA].filter(x => !setB.has(x)).concat([...setB].filter(x => !setA.has(x)))
);
// Результат: Set {'Toyota', 'Tesla'}
```

КОЛЕКЦІЯ MAP ТА ОПЕРАЦІЇ З ДАНИМИ

`Map` це сучасна структура даних у JavaScript, яка зберігає пари «ключ-значення». На відміну від звичайних об'єктів, де ключами можуть бути лише рядки або символи, у `Map` ключем може виступати **будь-який тип даних**: числа, об'єкти, масиви та навіть функції.

1. Основні методи керування даними

Для роботи з елементами `Map` використовується набір стандартних методів, які забезпечують високу швидкість доступу:

- **`set(key, value)`**: Додає новий елемент або оновлює існуючий за ключем. Повертає саму колекцію, що дозволяє будувати ланцюжки (chaining).
- **`get(key)`**: Повертає значення за ключем або `undefined`, якщо ключа не існує.
- **`has(key)`**: Перевіряє наявність ключа в колекції (повертає `true/false`).
- **`delete(key)`**: Видаляє запис за вказаним ключем.

- **`clear()`**: Повністю очищує колекцію.
- **`size`**: Властивість (не метод), що повертає актуальну кількість записів.

2. Складні типи даних як ключі

Однією з найпотужніших особливостей `Map` є можливість використовувати об'єкти як ідентифікатори. Це дозволяє пов'язувати додаткові дані з об'єктом, не змінюючи сам об'єкт.

Приклад:

```
JavaScript
const student = { id: 202401, name: "Олексій" };
const progress = new Map();

// Використовуємо об'єкт студента як ключ
progress.set(student, {
    completedSteps: ["Реєстрація", "Подання
документів"],
    lastActive: "2024-05-20"
});

console.log(progress.get(student).lastActive); //
"2024-05-20"
```

3. Ітерація та перебір даних

`Map` завжди зберігає порядок додавання елементів, що важливо для передбачуваного виведення даних на сторінку.

- **`map.keys()`**: Повертає ітератор усіх ключів.
- **`map.values()`**: Повертає ітератор усіх значень.
- **`map.entries()`**: Повертає ітератор пар `[key, value]` (використовується за замовчуванням у `for...of`).
- **`map.forEach((value, key) => { ... })`**: Виконує функцію для кожного елемента.

4. Трансформація даних (Map ↔ Array)

Для виконання складних операцій (наприклад, фільтрації або сортування) `Map` часто конвертують у масив і навпаки.

- **З `Map` у масив**: Використовується оператор розширення `[...]` або `Array.from()`.

```
JavaScript
const mapAsArray = [...myMap]; // Масив вигляду [[key1,
val1], [key2, val2]]
```

- **З масиву у `Map`**: Конструктор `new Map()` приймає масив пар.

```
JavaScript
```

```
const newMap = new Map([
    ["Ford", 25000],
    ["Tesla", 45000]
]);
```

5. Обробка JSON-даних через Map

Оскільки формат JSON не підтримує структуру `Map` безпосередньо, при роботі з API або файлами використовується проміжний етап конвертації в масив:

Серіалізація: `const json = JSON.stringify([...myMap]);`
Десеріалізація: `const myMap = new Map(JSON.parse(jsonString));`

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

***Завдання 1.* Аналіз унікальності (Set)**

1. Створіть функцію, яка приймає два масиви (наприклад, "Список авто в наявності" та "Список авто під замовлення").
2. За допомогою `Set` знайдіть лише ті марки, які зустрічаються в обох списках одночасно (перетин множин).

***Завдання 2.* Порівняння складів автосалонів.**

1. Створіть дві множини `stockKyiv` та `stockLviv` із марками авто.
2. Знайдіть **спільні марки** (перетин), які представлені в обох містах.
3. Знайдіть **унікальні марки** для Києва (різниця), яких немає у Львові.
4. Виведіть результат у консоль у форматі: *"Спільні бренди: Ford, Toyota"*.

***Завдання 3.* Розумний кеш статусів (Map)**

1. Створіть систему, де ключем у Map є об'єкт Student.
2. Значенням у Map має бути масив об'єктів з "Дорожньої карти" (JSON), які цей студент уже виконав.
3. Реалізуйте метод, який дозволяє додати новий виконаний етап студенту, не зачіпаючи дані інших.

***Завдання 4.* Розумний фільтр автосалону**

1. Створіть `Map`, де ключами є назви марок авто, а значеннями −об'єкти з детальною інформацією (країна, рік заснування).
2. Напишіть функцію, яка приймає цю `Map` і за допомогою методів ітерації виводить у консоль лише ті бренди, які були засновані після 1950 року.
3. Продемонструйте видалення одного з брендів за ключем та виведіть оновлений розмір колекції через `.size`.

***Завдання 5.* Трансформація для збереження**

Створіть складну карту Map, де ключі − це ID етапів вступу, а значення − об'єкти з деталями.

*Напишіть код, який конвертує цю Map у формат JSON та зберігає у localStorage під назвою "user_progress".

*При завантаженні сторінки відновіть Map із localStorage

ДОДАТКОВЕ ЗАВДАННЯ

Сценарій - Інформаційна система моніторингу етапів вступу.

У цьому завданні ви маєте створити інтелектуальну систему, яка зберігає стан виконання етапів вступної кампанії для конкретного абітурієнта, використовуючи об'єкти як ключі.

Завдання 1. Створення та ініціалізація (set, size)

1. Створіть масив об'єктів `admissionSteps` на основі даних з файлу `road_bachelor.json` (мінімум 5 етапів).
2. Створіть колекцію `Map` з назвою `stepsRegistry`.
3. За допомогою циклу заповніть `Map`, де:
   - **Ключем** є сам об'єкт етапу.
   - **Значенням** є рядок зі статусом (наприклад: `"очікує"`, `"виконується"`, `"завершено"`).
4. Виведіть у консоль кількість зареєстрованих етапів, використовуючи властивість `.size`.

Завдання 2. Пошук та оновлення (get, has, set)

1. Напишіть функцію `updateStepStatus(stepObject, newStatus)`, яка:
   - Перевіряє, чи існує цей об'єкт у `stepsRegistry` за допомогою `.has()`.
   - Якщо існує − оновлює його статус через `.set()`.
   - Якщо не існує −виводить попередження: *"Етап не знайдено у дорожній карті"*.
2. Знайдіть у масиві об'єкт, що відповідає етапу "Подання заяв", та змініть його статус на `"завершено"`.
3. Виведіть актуальний статус цього етапу в консоль, звернувшись до `Map` через `.get()`.

Завдання 3. Моделювання робочого процесу (delete, clear)

1. Уявіть, що один із етапів (наприклад, "Додаткова сесія НМТ") було скасовано. Видаліть відповідний об'єкт із колекції `stepsRegistry`, використовуючи метод `.delete()`.
2. Виведіть оновлений розмір колекції.
3. Створіть функцію "Скидання прогресу", яка повністю очищує колекцію `stepsRegistry` за допомогою методу `.clear()` та виводить повідомлення про успішне очищення.

Завдання 4. Ітерація та звітність

1. Використовуючи метод `.forEach()` або цикл `for...of`, виведіть на сторінку (або в консоль) повний звіт у форматі:

   Етап: [Назва етапу] − СТАТУС: [Значення статусу]